

Progetto di reti Logiche

Funzione seno in virgola fissa

Fabrizio Giacona

Introduzione

Il progetto richiede la realizzazione di un modello hardware descritto in VHDL, capace di calcolare il seno di un angolo espresso in gradi interi nell'intervallo [0;359]. Il valore del seno è rappresentato in fixed point con segno su 10 bit (2 bit per la parte intera e 8 bit per la parte frazionaria).

Il modulo deve calcolare il seno di un angolo compreso tra 0 e 90 gradi, quindi utilizzare le identità goniometriche per calcolare il seno degli angoli tra 91 e 359 gradi. Il calcolo deve essere basato su una look-up table che riporta il seno degli angoli multipli di 8 e degli angoli 89 e 90, se l'angolo di ingresso è diverso da uno di quelli della look-up table, viene calcolato mediante interpolazione lineare.

Specifica

Il sistema riceve in ingresso un angolo in gradi codificato in binario naturale su 9 bit.

L'elaborazione viene suddivisa in tre fasi principali:

La prima fase consiste nella determinazione dell'angolo equivalente appartenente al primo quadrante, per tale scopo si applicano le identità trigonometriche.

La seconda fase consiste nel calcolo del seno dell'angolo equivalente, se il valore dell'angolo coincide con uno dei valori memorizzati nella look-up table, il risultato è ottenuto direttamente dalla memoria, altrimenti vengono individuati i due valori memorizzati più vicini all'angolo richiesto e viene eseguita un'interpolazione lineare.

La terza fase consiste nel correggere il segno del risultato, se l'angolo in ingresso nella prima fase era nell'intervallo [181;359] il seno ha valore negativo, di conseguenza bisogna invertire il segno del risultato, altrimenti il risultato rimane invariato.

Interfaccia del sistema

Ingressi:

- ANGLE: angolo espresso in gradi codificato in binario naturale su 9 bit, i valori ammessi sono compresi tra 0 e 359.
- CLK: segnale di clock del sistema.
- RST: segnale di reset sincrono.

Uscite:

- RESULT: valore del seno dell'angolo in ingresso codificato in fixed point Q2.8 su 10 bit.

Il sistema è completamente sincrono e opera sul fronte di salita del clock.

L'angolo di ingresso viene acquisito da un registro parallelo-parallelo all'ingresso del sistema, dopo l'acquisizione dell'ingresso, il dato attraversa i blocchi combinatori interni e il risultato viene memorizzato in un registro di uscita.

Il sistema introduce una latenza di due cicli di clock tra l'applicazione del dato sull'ingresso ANGLE e la disponibilità del corrispondente risultato sull'uscita RESULT.

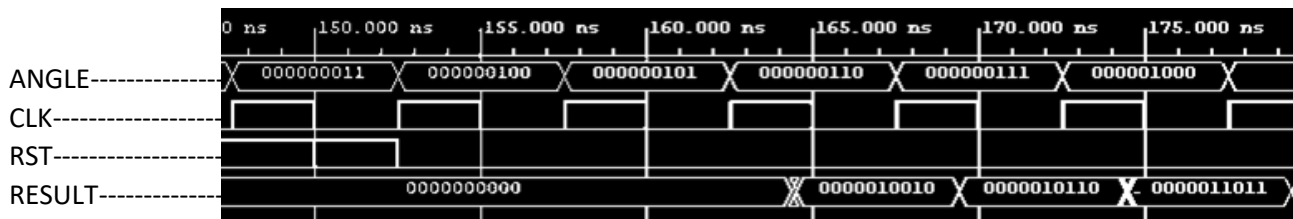


Figura 1 – diagramma temporale (waveform)

Il diagramma temporale riportato in figura mostra l'andamento del segnale **RESULT** al variare dei segnali **ANGLE** e **RST**. Si osserva che il valore corretto di **RESULT** è disponibile dopo una latenza di due cicli di clock dall'applicazione dell'angolo in ingresso, oltre al ritardo di propagazione dovuto alla logica del circuito.

Architettura del sistema

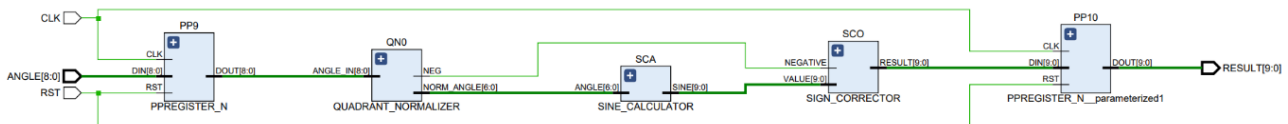
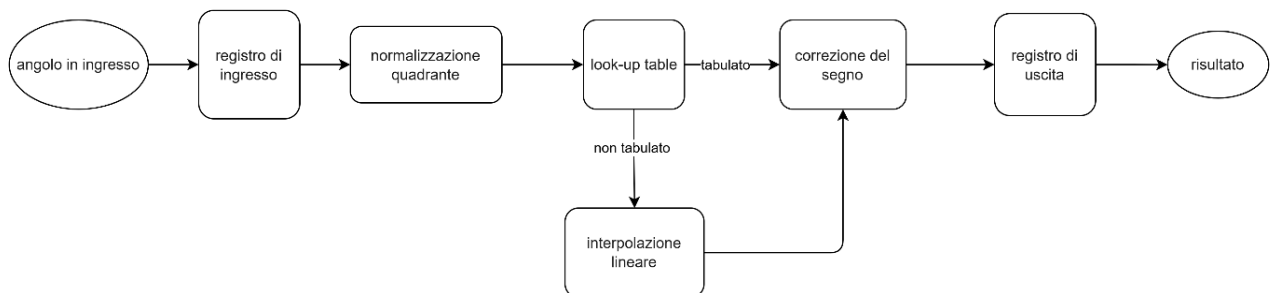


Figura 2 - Architettura Generale

Il sistema è organizzato come una pipeline a singolo stadio combinatorio compresa tra due registri.

Il percorso dei dati attraversa tre stadi funzionali in cascata. QUADRANT_NORMALIZER riduce l'angolo di ingresso [0;359] al primo quadrante [0;90] applicando le identità trigonometriche, producendo NORM_ANGLE e il flag NEGATIVE. SINE_CALCULATOR riceve NORM_ANGLE e calcola il valore del seno tramite look-up table e interpolazione lineare, restituendo il risultato in formato fixed point Q2.8. SIGN_CORRECTOR applica la negazione in complemento a due quando NEGATIVE è alto, producendo il valore finale con segno corretto.

Le operazioni aritmetiche sono costruite strutturalmente a partire da full adder, aggregato nel componente generico ADD_SUB_N mediante il costrutto *generate*. La generalizzazione del codice è massimizzata tramite i parametri *generic* applicati in PPREGISTER_N e ADD_SUB_N.



Modulo QUADRANT_NORMALIZER

segnale	direzione	tipo	codifica
ANGLE	in	std_logic_vector	9 bit, binario naturale
NORM_ANGLE	out	std_logic_vector	7 bit, binario naturale
NEGATIVE	out	std_logic	1 bit

Il modulo QUADRANT_NORMALIZER riceve l'angolo grezzo [0;359] e lo riconduce al primo quadrante [0;90] applicando le identità trigonometriche. Produce NORM_ANGLE e il flag NEGATIVE, che segnala se il seno dell'angolo originale è negativo (utile per l'ultimo modulo).

Per determinare quale identità trigonometrica va applicata per la normalizzazione dell'angolo si effettuano confronti con le soglie di quadrante, realizzate tramite sottrazioni (ADD_SUB_N) usando il carry-out come indicatore di maggiore-uguale, senza usare comparatori espliciti. Una volta individuato il quadrante è necessaria una sottrazione (ADD_SUB_N) per ottenere l'angolo normalizzato, i termini di tale sottrazione dipendono solo dal quadrante dell'angolo di ingresso.

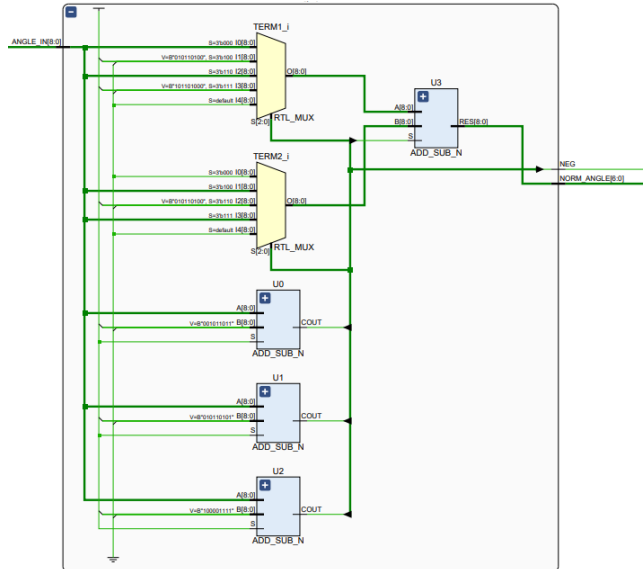


Figura 3 - QUADRANT_NORMALIZER

$$x \in [91; 180] \rightarrow \sin(x) = \sin(180 - x)$$

$$x \in [181; 270] \rightarrow \sin(x) = -\sin(x - 180)$$

$$x \in [271; 360] \rightarrow \sin(x) = -\sin(360 - x)$$

Modulo SINE_CALCULATOR

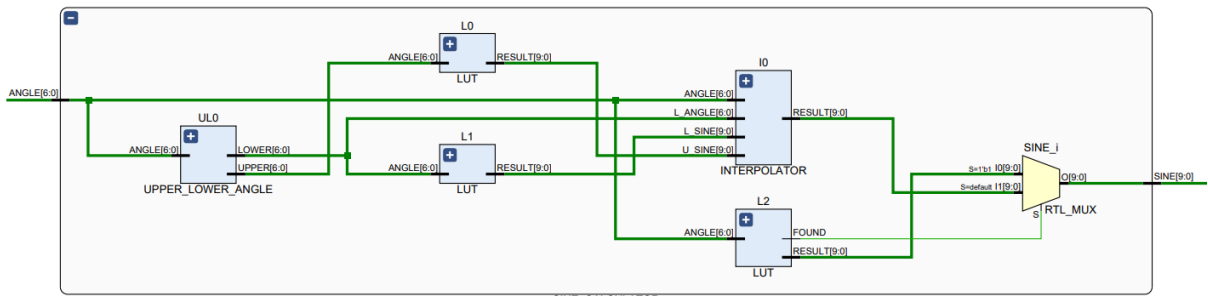


Figura 4 modulo SINE_CALCULATOR

segnale	direzione	tipo	codifica
ANGLE	in	std_logic_vector	7 bit binario naturale
SINE	out	std_logic_vector	10 bit fixed point Q2.8

Il modulo calcola $|\sin(ANGLE)|$, se l'angolo in ingresso è già presente in tabella allora il risultato viene direttamente dall'uscita della look-up table, altrimenti il risultato viene dal risultato dell'interpolazione: il componente UPPER_LOWER_ANGLE individua i due angoli tabulati più vicini utili per l'interpolazione lineare, entrambi andranno in ingresso in delle look-up table in modo da avere tutti i valori necessari per eseguire l'interpolazione lineare.

Modulo UPPER_LOWER_ANGLE

segnale	direzione	tipo	codifica
ANGLE	in	std_logic_vector	7 bit binario naturale
LOWER	out	std_logic_vector	10 bit fixed point Q2.8
UPPER	out	std_logic_vector	10 bit fixed point Q2.8

Il modulo individua i due angoli tabulati più vicini (multipli di 8) attraverso una divisione per 8 dell'angolo in ingresso corrispondente a uno shift logico a destra di 3 bit (realizzato per semplice connessione di segnali, senza alcuna logica attiva).

SLOT <= ANGLE (6 **downto** 3);

Il risultato della divisione permette di ottenere gli angoli superiore e inferiore attraverso un costrutto *case*.

Modulo LUT

segnale	direzione	tipo	codifica
ANGLE	in	std_logic_vector	7 bit binario naturale
RESULT	out	std_logic_vector	10 bit fixed point Q2.8
FOUND	out	std_logic	1 bit

Il modulo implementa una memoria di sola lettura combinatoria contenente i valori del seno per gli angoli campionati (multipli di 8 + angoli 89 e 90). Se l'angolo non è presente in tabella RESULT assume valore "111111111", il segnale FOUND indica se l'angolo in ingresso è presente in tabella.

nota: $\sin(88^\circ)$ e $\sin(89^\circ)$ superano la soglia di arrotondamento, entrambi valgono "0100000000".

Modulo INTERPOLATOR

segnale	direzione	tipo	codifica	descrizione
ANGLE	in	std_logic_vector	7 bit binario naturale	Angolo richiesto [0;90]
L_ANGLE	in	std_logic_vector	7 bit binario naturale	Angolo tabulato inferiore
L_SINE	in	std_logic_vector	10 bit fixed point Q2.8	$\sin(L_ANGLE)$
U_SINE	in	std_logic_vector	10 bit fixed point Q2.8	$\sin(U_ANGLE)$
RESULT	out	std_logic_vector	10 bit fixed point Q2.8	Risultato interpolazione

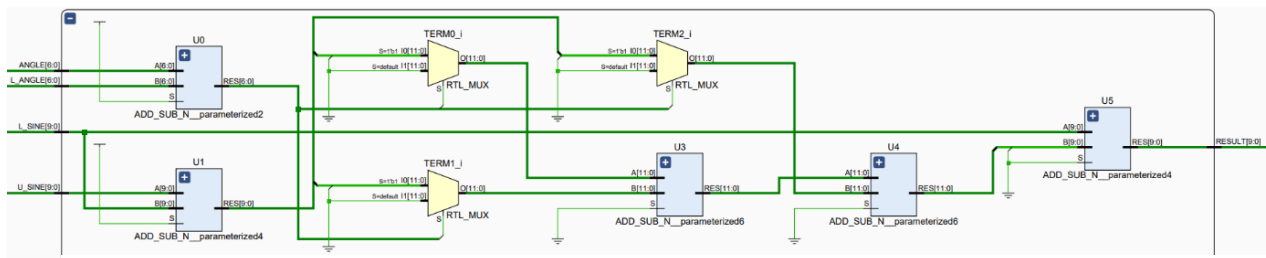


Figura 5 modulo INTERPOLATOR

Il modulo realizza lo stadio aritmetico dell'interpolazione lineare.

RESULT viene calcolato come: $RESULT = L_{SINE} + \frac{DIFF \cdot \Delta}{8}$

dove $\Delta = ANGLE - L_ANGLE$ può assumere valori compresi tra 0 e 7, è quindi possibile realizzare il termine moltiplicativo tramite somma di 3 versioni shiftate di $DIFF = U_SINE - L_SINE$.

È stata valutata l'adozione di una struttura Carry Save per la somma dei tre termini. Tuttavia, poiché il numero di operandi è limitato a tre e la larghezza dei dati è di soli 12 bit, il beneficio temporale risulta trascurabile rispetto alla complessità aggiuntiva introdotta. Si è pertanto scelto di utilizzare due sommatore convenzionali in cascata, soluzione più semplice e che permette maggiore riuso.

Tutte le rimanenti somme e sottrazioni vengono svolte tramite il componente ADD_SUB_N, mentre lo shift logico per la divisione per 8 viene realizzato per semplice connessione di segnali.

A causa dell'interpolazione lineare si stima un errore teorico, il cui valore massimo può essere stimato come:

$e_{\max} = \frac{h^2}{8} \cdot \max(|f''(x)|)$ dove h rappresenta l'ampiezza dell'intervallo di interpolazione

(in questo caso $h = 8 = 0.1396$ rad e $f(x) = \sin(x)$ quindi $\max(|f''(x)|) = 1$). Si ottiene quindi:

$e_{\max} \approx 0.00244$, il bit meno significati in fixed point 2.8 vale $2^{-8} \approx 0.00391$, dunque l'errore massimo vale

$\frac{e_{\max}}{LSB} \approx 0.624$, nonostante l'errore sia inferiore a 1 LSB, la quantizzazione del risultato in fixed point può tradurre l'errore in una differenza di 1 LSB rispetto al valore ideale della funzione seno.

L'approccio adottato permette quindi di ottenere una buona accuratezza limitando significativamente le dimensioni della memoria necessaria e la complessità circuitale.

Modulo SIGN_CORRECTOR

segnale	direzione	tipo	codifica
VALUE	in	std_logic_vector	10 bit fixed point Q2.8
NEGATIVE	in	std_logic	1 bit
RESULT	out	std_logic_vector	10 bit fixed point Q2.8

Il modulo usa un sommatore per complementare a due il valore in ingresso, in particolare se il segnale di NEGATIVO vale '0', il sommatore esegue la somma $0 + \text{VALUE}$, quindi il risultato è uguale al valore in ingresso, altrimenti se NEGATIVO vale '1' il sommatore esegue la somma $0 + (-\text{VALUE})$ invertendo quindi il segno.

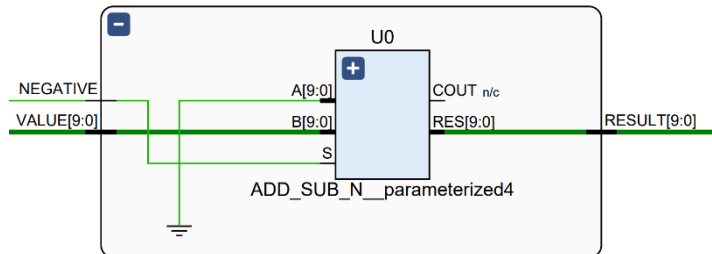


Figura 6 modulo SIGN_CORRECTOR

Modulo ADD_SUB_N

segnale	direzione	tipo	codifica	descrizione
A	in	std_logic_vector	N bit binario naturale	Primo operando
B	in	std_logic_vector	N bit binario naturale	Secondo operando
S	in	std_logic	1 bit	Selezione addizione ('0') sottrazione ('1')
RES	out	std_logic_vector	N bit binario naturale	Risultato
COUT	out	std_logic	1 bit	Carry-out

Il modulo realizza un sommatore/sottrattore ripple carry generalizzato a N bit.

Verifica

La verifica si basa sulla simulazione di tipo timing post-implementation, che incorpora tutti i ritardi reali dei percorsi logici. La strategia di verifica adottata ha previsto una validazione gerarchica del progetto, ciascun modulo principale è stato inizialmente verificato mediante un testbench dedicato, successivamente è stata effettuata la verifica del componente completo.

Per esercitare il percorso critico del sistema, i casi di test includono angoli che attivano le catene combinatorie più lunghe, ovvero quelli che richiedono interpolazione lineare e inversione del segno.

Il periodo di clock è stato determinato empiricamente attraverso prove ripetute: partendo da un valore conservativo, il vincolo è stato progressivamente ridotto fino a individuare il minimo periodo per cui la simulazione timing post-implementation non riporta errori (5 ns).

Test-bench

Il test-bench del sistema top-level SINE_FUNCTION istanzia il componente sotto test e gli applica in sequenza tutti i 360 ingressi validi [0;359] attraverso un costrutto *for loop*.

Il generatore di clock è realizzato mediante un processo dedicato con un loop infinito che commuta il segnale CLK ogni mezzo periodo.

Casi d'uso

I casi di test coprono l'intero dominio di ingresso e possono essere distinti in quattro categorie, isolando i percorsi hardware al fine di identificare più facilmente gli errori:

- Angoli esatti della LUT
- Angoli interpolati nel I quadrante
- Angoli interpolati nel II quadrante
- Angoli interpolati nel III e IV quadrante

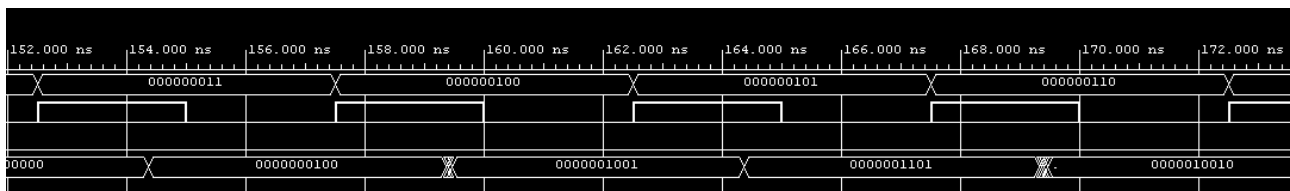


Figura 7 – waveform della simulazione timing post-implementation